

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №7
з курсу «Хмарні технології та великі дані»
на тему:
«Fan-Out (Демультіплексор)»

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	3
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	5
Висновок	10
Код програми	11

Тема

Паралельний розподіл обчислювального навантаження та конкурентна обробка завдань за допомогою патерну Fan-Out (Демальтиплексор / Worker Pool).

Мета

Дослідження принципу функціонування патерну Fan-Out у контексті паралельної обробки асинхронних операцій у середовищі TypeScript. Розробка надійної типізованої структури пулу воркерів `fanOut<T, R>` з підтримкою динамічної черги завдань, масштабуванням продуктивності за рахунок пулу паралельних обробників, гарантією семантики «суворо одноразової обробки» (No Duplicate Processing), коректною ізоляцією збоїв через тип `Result<T, E>` та реалізацією механізму плавного завершення (Graceful Shutdown).

Вхідні дані

Параметр	Значення	Призначення
<code>process</code>	<code>TaskProcessor<T, R></code>	Асинхронна функція-обробник, що виконує цільову роботу над елементом <code>task</code> типу <code>T</code> та повертає результат <code>Promise<R></code> .
<code>workers</code>	Ціле додатне число (<code>number > 0</code>)	Розмір пулу: кількість одночасно працюючих конкурентних воркерів.
<code>task</code>	Параметризований тип <code>T</code>	Одиниця роботи (завдання), що передається через метод <code>submit(task)</code> до внутрішньої черги.
<code>Result<T, E></code>	<code>{ ok: true; value: T } { ok: false; error: E }</code>	Дискримінантне об'єднання для безпечної передачі результату успішного виконання або винятку без падіння пулу.
<code>queue</code>	Внутрішній масив <code>T[]</code>	Буфер черги завдань (FIFO), з якого вільні воркери вилучають задачі для виконання.

Завдання

- Розробити універсальну типізовану функцію `fanOut<T, R>(process, workers): FanOutPool<T, R>` мовою TypeScript з валідацією кількості воркерів (`workers > 0`).
- Створити інтерфейс пулу `FanOutPool<T, R>`, що надає три методи взаємодії: `submit(task)` для додавання задач, `results()` для зчитування результатів через `AsyncIterable`, та `close()` для ініціалізації завершення роботи.

- Організувати конкурентний цикл роботи воркерів (`workerLoop`), які незалежно вилучають завдання з черги (`queue.shift()`) та паралельно їх обробляють.
- Забезпечити сувору семантику відсутності дублювання (No Duplicate Processing): жодна задача не повинна виконуватися повторно або губитися під час розподілу між конкурентними обробниками.
- Реалізувати безпечну ізоляцію помилок (Error Isolation): перехоплення винятків усередині воркера через блок `try/catch` із фіксацією збою як `{ ok: false, error }`, що запобігає аварійній зупинці всього пулу.
- Реалізувати механізм Graceful Shutdown: після виклику `close()` блокувати прийом нових завдань викликом `submit()`, гарантовано дочекатися завершення поточної черги та активних воркерів, після чого коректно завершити асинхронний ітератор результатів.
- Розробити демонстраційний набір перевірок (`fanOut-demo.ts`) без використання сторонніх тестових бібліотек для верифікації паралелізму, цілісності даних, зупинки та ізоляції помилок.

Критерії виконання роботи

- *Паралелізм та прискорення*: Збільшення кількості воркерів ($1 \rightarrow 2 \rightarrow 4$) для незалежних повільних операцій пропорційно скорочує загальний час обробки.
- *Цілісність обробки даних*: Кількість завершених завдань суворо дорівнює кількості поданих завдань ($N_{completed} = N_{submitted}$), кількість втрачених і дубльованих задач дорівнює нулю.
- *Коректне закриття (Graceful Shutdown)*: Виклик `pool.submit()` після `pool.close()` гарантовано повертає помилку, водночас усі раніше додані завдання коректно обробляються до кінця, після чого стрім `results()` плавно завершується.
- *Ізоляція збоїв*: Збій або відхилення окремого завдання не руйнує цикл обробника; воркери продовжують функціонувати та опрацьовувати наступні завдання з черги.

Теоретичні відомості

Fan-Out (Демальтиплексор / Worker Pool) — це патерн паралельної та реактивної обробки даних, суть якого полягає в декомпозиції загального потоку завдань і їх паралельному розподілі між декількома незалежними обробниками (воркерами).

У середовищах із моделлю Event Loop (зокрема Node.js) тривалі операції введення-виведення (або асинхронні звернення до зовнішніх сервісів) при послідовному виконанні виконуються сумарний час $T_{total} = \sum t_i$. Застосування патерну Fan-Out дозволяє паралелізувати незалежні операції до N воркерів, зменшуючи теоретичний час обробки до величини:

$$T_{total} \approx \frac{\sum t_i}{N}$$

Ключовими архітектурними аспектами надійної реалізації Fan-Out є:

- *Буферизація та черга*: Черга завдань слугує буфером, що вирівнює сплески навантаження (load-leveling). Воркери конкурують за наступне завдання, вилучаючи його з черги атомарно.
- *Ізоляція помилок*: В аварійних умовах падіння одного виклику не повинно спричиняти паніку пулу або переривання інших незалежних задач. Для цього результати операцій упаковуються у контейнер `Result<T, E>`.
- *Graceful Shutdown*: При завершенні роботи компонента неприпустимо обривати незавершені задачі. Пул переходить у стан "закриття" (`closed = true`), припиняє прийом нових задач і завершує роботу лише після того, як черга стає порожньою (`queue.length = 0`) та всі воркери завершили свою поточну роботу (`activeWorkers = 0`).

Структура програмної реалізації

Рішення складається з наступних модулів:

- *src/fanOut.ts* — ядро патерну Fan-Out:
 - Інтерфейс `FanOutPool<T, R>` та типізація результатів `Result<T, E>`.
 - Функція `fanOut(process, workers)`, що ініціалізує чергу, сховище результатів та запускає конкурентні цикли `workerLoop`.
 - Реалізація генератора `results()`, що видає оброблені результати в міру їх готовності через протокол `AsyncIterable`.
- *fanOut-demo.ts* — автономний сценарій комплексної верифікації, що містить 4 тестові сценарії з детальним логуванням:
 - `parallelismDemo` — перевірка часу виконання при зміні конфігурації воркерів (1, 2, 4).
 - `noDuplicateDemo` — перевірка коректності розподілу 12 задач без втрат і дублювань.
 - `gracefulShutdownDemo` — перевірка поведінки пулу при виклику `close()` під час активної роботи. `errorHandlingDemo` — перевірка життєздатності воркерів при виникненні помилок у завданнях.

Тестування програмної реалізації

Верифікація коректності реалізації пулу Fan-Out виконувалася за допомогою виконання скрипта `npm run demo`.

Дослід 1. Дослідження конкурентності та паралелізму

Вихідні дані тесту: 4 незалежні асинхронні задачі із фіксованою затримкою по 500 мс кожна. Тестування проводиться для трьох конфігурацій: 1 воркер, 2 воркери та 4 воркери.

```

TEST 1 – PARALLELISM
=====
Goal: increasing the number of workers should reduce the total time for independent slow tasks.
[START] Worker 1 started Task 1
[DONE ] Worker 1 completed Task 1 -> result=2
[START] Worker 1 started Task 2
[DONE ] Worker 1 completed Task 2 -> result=4
[START] Worker 1 started Task 3
[DONE ] Worker 1 completed Task 3 -> result=6
[START] Worker 1 started Task 4
[DONE ] Worker 1 completed Task 4 -> result=8

--- Run with 1 worker(s) ---
Results:
  Task 1 -> 2
  Task 2 -> 4
  Task 3 -> 6
  Task 4 -> 8
Total time: approximately 2066 ms
[START] Worker 1 started Task 1
[START] Worker 2 started Task 2
[DONE ] Worker 1 completed Task 1 -> result=2
[START] Worker 1 started Task 3
[DONE ] Worker 2 completed Task 2 -> result=4
[START] Worker 2 started Task 4
[DONE ] Worker 1 completed Task 3 -> result=6
[DONE ] Worker 2 completed Task 4 -> result=8

```

```

--- Run with 2 worker(s) ---
Results:
  Task 1 -> 2
  Task 2 -> 4
  Task 3 -> 6
  Task 4 -> 8
Total time: approximately 1074 ms
[START] Worker 1 started Task 1
[START] Worker 2 started Task 2
[START] Worker 3 started Task 3
[START] Worker 4 started Task 4
[DONE ] Worker 1 completed Task 1 -> result=2
[DONE ] Worker 2 completed Task 2 -> result=4
[DONE ] Worker 3 completed Task 3 -> result=6
[DONE ] Worker 4 completed Task 4 -> result=8

--- Run with 4 worker(s) ---
Results:
  Task 1 -> 2
  Task 2 -> 4
  Task 3 -> 6
  Task 4 -> 8
Total time: approximately 534 ms

```

Рис.1 –Конкурентність та паралелізм в патерні fan-out

При використанні 1 воркера задачі виконувалися строго послідовно одна за одною. Загальний час склав ~2018 мс, що відповідає очікуваним 4×500 мс + накладні витрати. Далі збільшення пулу до 2 воркерів дозволило обробляти задачі парами. Час виконання скоротився рівно вдвічі — до ~1012 мс (2×500 мс). Конфігурація із 4 воркерами запустила всі 4 задачі одночасно. Сумарний час впав до ~508 мс, що відповідає тривалості виконання однієї задачі. Експеримент підтвердив лінійну ефективність паралелізації: пул асинхронних воркерів усуває блокування черги та масштабує пропускну здатність системи.

Дослід 2. Перевірка цілісності обробки даних

Вихідні дані тесту: 12 задач із випадково-детермінованими затримками (50 ... 125 мс) подаються у чергу пулу із 4 воркерів.

```
=====
TEST 2 – DATA SAFETY / NO DUPLICATE PROCESSING
=====
Goal: every submitted task must be processed exactly once.
[DONE] Task 4 -> processed 1 time(s)
[DONE] Task 1 -> processed 1 time(s)
[DONE] Task 2 -> processed 1 time(s)
[DONE] Task 3 -> processed 1 time(s)
[DONE] Task 5 -> processed 1 time(s)
[DONE] Task 6 -> processed 1 time(s)
[DONE] Task 8 -> processed 1 time(s)
[DONE] Task 9 -> processed 1 time(s)
[DONE] Task 7 -> processed 1 time(s)
[DONE] Task 12 -> processed 1 time(s)
[DONE] Task 10 -> processed 1 time(s)
[DONE] Task 11 -> processed 1 time(s)

-----
TASK PROCESSING COUNTS
-----
Task 1   -> 1
Task 2   -> 1
Task 3   -> 1
Task 4   -> 1
Task 5   -> 1
Task 6   -> 1
Task 7   -> 1
Task 8   -> 1
Task 9   -> 1
Task 10  -> 1
Task 11  -> 1
Task 12  -> 1

Submitted tasks : 12
Completed tasks : 12
Missing tasks    : 0
Duplicate tasks  : 0
=====
```

Рис.2 –Цілісність обробки даних в патерні fan-out

Усі 12 переданих завдань були успішно вилучені з черги та виконані. Кількість зафіксованих повторних обробок (дублікатів) дорівнює 0, а кількість пропущених задач — 0. Це свідчить про коректну роботу механізму атомарного забору задач із внутрішньої черги (`queue.shift()`) у циклі `workerLoop`, що унеможливорює стан гонитви (`race conditions`) між конкуруючими асинхронними воркерами.

Дослід 3. Перевірка контрольованого завершення

Вихідні дані тесту: До пулу з 2 воркерів надсилаються задачі 1–4, викликається `close()`, після чого робиться спроба додати задачу 5.

```

=====
TEST 3 – GRACEFUL SHUTDOWN
=====
Goal: shutdown should stop new submissions but allow already submitted tasks to finish.
Submitting tasks...
[SUBMIT] Task 1 accepted
[SUBMIT] Task 2 accepted
[SUBMIT] Task 3 accepted
[SUBMIT] Task 4 accepted

Calling close()...
[SHUTDOWN] No more tasks will be accepted.
[EXPECTED] Task 5 was rejected because the system is shutting down.

Workers are still processing submitted tasks...
[DONE] Task 1 completed
[DONE] Task 2 completed
[DONE] Task 3 completed
[DONE] Task 4 completed
[SHUTDOWN] Queue is empty and all workers have finished.
[RESULTS] Result stream completed.

Submitted before shutdown : 4
Completed                  : 4
Submitted after shutdown  : rejected
PASS: graceful shutdown completed correctly.

```

Рис.3 –Контрольоване завершення в патерні fan-out

Метод `pool.close()` перевів внутрішній стан у `closed = true`. Спроба викликати `submit(5)` одразу викинула передбачуваний виняток (`fanOut is closed; no new tasks can be submitted`), підтвердивши захист від нових завдань під час зупинки. Воркери не перервали роботу аварійно: задачі 1–4 були повністю оброблені. Асинхронний ітератор результатів `results()` коректно повернув керування (`return`) тільки тоді, коли зійшлися три умови: встановлено прапорець закриття, черга порожня та всі воркери завершили обробку (`closed && activeWorkers = 0 && queue.length = 0`). Це запобігає «зависанню» споживачів стріму.

Дослід 4. Перевірка відмовостійкості та ізоляції помилок

Вихідні дані тесту: Пул із 3 воркерів обробляє 6 завдань, при цьому Task 2 навмисно завершується помилкою (`{ ok: false, error: 'intentional test error' }`).

```

=====
TEST 4 – ERROR ISOLATION
=====
Goal: one task failure must not stop the rest of the worker pool.
[START] Worker 1 started Task 1
[START] Worker 2 started Task 2
[START] Worker 3 started Task 3
[DONE ] Worker 3 completed Task 3 -> result=9
[START] Worker 1 started Task 4
[DONE ] Worker 1 completed Task 1 -> result=3
[START] Worker 2 started Task 5
[ERROR] Worker 2 failed Task 2 -> intentional test error
[START] Worker 3 started Task 6
[DONE ] Worker 1 completed Task 4 -> result=12
[DONE ] Worker 3 completed Task 6 -> result=18
[DONE ] Worker 2 completed Task 5 -> result=15

-----
ERROR SUMMARY
-----
Successful tasks : 5
Failed tasks     : 1
Worker pool      : still operational
PASS: one task failure did not terminate the worker pool.

```

Рис.4 –Відмовостійкість та ізоляція в патерні fan-out

Збій на завданні Task 2 був зафіксований та локалізований у Worker 2. Воркер не вийшов із циклу workerLoop, а перехопив помилку, зберіг статусний результат у чергу видачі й одразу взяв наступне завдання (Task 4). Решта воркерів (Worker 1 та Worker 3) продовжували паралельне виконання своїх завдань без жодних затримок чи збоїв. Загальний результат: 5 задач завершено успішно, 1 помилку ізольовано, працездатність пулу повністю збережена.

Висновок

У ході виконання лабораторної роботи було досліджено теоретичні основи та практичні аспекти побудови високонавантажених асинхронних конвеєрів обробки завдань за патерном Fan-Out (Демальтиплексор / Worker Pool) мовою TypeScript.

Було розроблено типізовану систему fanOut, яка реалізує безпечний розподіл завдань між фіксованим пулом воркерів через неблокуючу чергу. Експериментально підтверджено суттєве скорочення часу обробки пачки операцій при збільшенні кількості воркерів. Реалізований механізм ізоляції помилок на основі патерну Result<T, E> забезпечив неперервність обчислювального процесу у разі одиничних збоїв. Застосування процедури плавного закриття (Graceful Shutdown) дозволило гарантувати цілісність обробки вже прийнятих даних та унеможливило втрату завдань при зупинці системи.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_7 та має повну інструкцію запуску даної програми.